

实验一：NLP 前置技术——Python 环境搭建

目录

1	实验目的	2
2	实验环境	2
3	实验内容与步骤	3
3.1	Python 环境及依赖库配置	3
3.2	Jupyter Notebook 环境配置	3
4	中文分词与词频统计实验	4
4.1	代码与运行结果	4
4.2	结果说明	4
5	英文分词实验	4
5.1	代码与运行结果	4
5.2	结果说明	5
6	实验中遇到的问题及解决方法	5
6.1	jieba 模块缺失与安装检查	5
6.2	NLTK 数据资源缺失	5
7	实验结果分析	6
8	思考与分析	7
8.1	为什么中文不能简单使用空格进行分词?	7
8.2	为什么标点符号会出现在分词结果中?	7
8.3	如何删除标点符号?	7
9	实验总结	7

1 实验目的

本实验主要完成自然语言处理课程所需 Python 环境的搭建，并熟悉常用 NLP 工具的基本使用方法。通过本次实验，需要掌握 Python 运行环境及相关依赖库的配置方法，学会使用 Jupyter Notebook 进行代码编写和运行，并完成简单的中文和英文文本处理任务。

具体实验目标如下：

1. 掌握 Python 及 Anaconda 环境的基本使用方法；
2. 掌握常用 Python 第三方库的安装与管理方法；
3. 熟悉 Jupyter Notebook 的基本操作；
4. 掌握 jieba 中文分词工具的基本使用方法；
5. 掌握 NLTK 英文分词工具的基本使用方法；
6. 使用 Counter 对分词结果进行简单的词频统计；
7. 学会分析并解决 Python 环境配置和资源缺失等常见问题。

2 实验环境

本次实验使用 Anaconda 提供的 Python 环境，在 Jupyter Notebook 中编写和运行程序。主要依赖包括 NumPy、pandas、Matplotlib、scikit-learn、jieba 和 NLTK。

实验开始后，通过导入相关库并打印版本信息，对当前环境进行验证。代码和实际输出见图1。

```
In [4]: import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
import jieba
import nltk

print('Python:', sys.version)
print('numpy:', np.__version__)
print('pandas:', pd.__version__)
print('sklearn:', sklearn.__version__)
print('jieba:', jieba.__version__)
print('nltk:', nltk.__version__)

Python: 3.14.6 | packaged by Anaconda, Inc. | (main, Jul 9 2026, 14:29:05) [MSC
v.1942 64 bit (AMD64)]
numpy: 2.4.6
pandas: 3.0.3
sklearn: 1.9.0
jieba: 0.42.1
nltk: 3.10.0
```

图 1: Python 环境与相关库的实际版本检查

截图中的 Python 版本为 3.14.6，jieba 版本为 0.42.1，NLTK 版本为 3.10.0，相关库均已成功导入。教师 Notebook 中的环境创建示例使用 Python 3.10，本报告记录的实际版本以图中输出为准。

3 实验内容与步骤

3.1 Python 环境及依赖库配置

首先检查 Python 和 Conda 环境是否已经正确安装，可以在命令行中输入：

Listing 1: Python 与 Conda 版本检查

```
1 python --version
2 conda --version
3
```

根据实验要求，可以通过 pip 安装自然语言处理所需的常用 Python 库，例如：

Listing 2: 相关依赖库安装

```
1 pip install numpy pandas matplotlib scikit-learn jieba nltk jupyter ipykernel
2
```

安装完成后，可以使用如下命令查看当前环境中已经安装的 Python 软件包：

```
1 pip list
2
```

通过上述步骤，为后续中文分词、英文分词以及其他自然语言处理实验建立基础运行环境。

3.2 Jupyter Notebook 环境配置

完成 Python 及相关第三方库的安装后，本实验使用 Jupyter Notebook 作为开发环境。

为了保证 Jupyter Notebook 使用正确的 Python 环境，可以将当前环境注册为 Jupyter 内核：

Listing 3: Jupyter 内核注册

```
1 python -m ipykernel install --user --name nlp --display-name "Python (NLP)"
2
```

之后可以通过以下命令启动 Jupyter Notebook：

```
1 jupyter notebook
2
```

启动完成后，浏览器会自动进入 Notebook 页面，即可进行代码编写与运行。

4 中文分词与词频统计实验

4.1 代码与运行结果

本实验使用 jieba 对给定中文文本进行分词，再使用 Counter 统计分词列表中各词元的出现次数。代码、分词结果和前 10 项词频见图2。

```
In [5]: import jieba
        from collections import Counter

        text = '自然语言处理是人工智能皇冠上的明珠，Python是自然语言处理常用的编程工具。'
        words = jieba.lcut(text)
        print('分词结果: ', words)

        word_freq = Counter(words)
        print('词频统计: ', word_freq.most_common(10))
```

Building prefix dict from the default dictionary ...
Dumping model to file cache C:\Users\MRFF6E~1.RIG\AppData\Local\Temp\jieba.cache
Loading model cost 1.160 seconds.
Prefix dict has been built successfully.

分词结果: ['自然语言', '处理', '是', '人工智能', '皇冠', '上', '的', '明珠',
, ' ', 'Python', '是', '自然语言', '处理', '常用', '的', '编程', '工具', '。']
词频统计: [('自然语言', 2), ('处理', 2), ('是', 2), ('的', 2), ('人工智能', 1),
('皇冠', 1), ('上', 1), ('明珠', 1), (' ', 1), ('Python', 1)]

图 2: jieba 中文分词与词频统计的运行结果

4.2 结果说明

分词结果中，“自然语言”和“处理”被划分为独立词元，中英混合文本中的 Python 也被保留。“自然语言”“处理”“是”“的”均出现 2 次，与原句中的重复情况一致。逗号和句号同样出现在分词列表中，且逗号进入了前 10 项词频结果，说明程序尚未进行标点过滤。

5 英文分词实验

5.1 代码与运行结果

本实验使用 NLTK 提供的 word_tokenize() 对给定英文句子进行分词。程序和实际输出见图3。

```
In [8]: import nltk
nltk.download('punkt')
from nltk.tokenize import word_tokenize

sentence = 'Natural language processing is a fascinating field.'
print(word_tokenize(sentence))

[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\Mr.Right\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
['Natural', 'language', 'processing', 'is', 'a', 'fascinating', 'field', '.']
```

图 3: NLTK 英文分词的运行结果

5.2 结果说明

程序将英文句子划分为 7 个单词和一个句点，句点作为独立词元保留。截图中的资源提示表明 punkt 已存在且处于最新状态，随后分词程序正常输出词元列表。

6 实验中遇到的问题及解决方法

6.1 jieba 模块缺失与安装检查

初次导入 jieba 时，程序提示当前环境无法找到该模块。处理这一问题时，需要保证 jieba 安装在 Jupyter Notebook 实际使用的 Python 环境中，而不是其他环境。

实验中保留的安装命令如图4所示。

```
In [ ]: pip install jieba
```

图 4: jieba 安装命令记录

这张截图的单元格执行编号为空，未展示安装输出，因此仅作为安装命令记录。后续图1显示 jieba 能够成功导入，图2也保存了正常分词结果，可以据此确认后续运行时该模块已可用。

6.2 NLTK 数据资源缺失

在进行英文分词时，遇到了 punkt_tab 资源无法找到的提示。NLTK 软件包能够导入，并不代表分词所需的数据资源已经齐全，因此需要另行检查和补充资源。

资源下载过程见图5。

```
In [7]: import nltk
```

sers/Mr.Right/Downloads/NLP实验1_2427030061_卿浩磊.html

11:14

NLP实验1_2427030061_卿浩磊

```
nltk.download('punkt')  
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to  
[nltk_data]   C:\Users\Mr.Right\AppData\Roaming\nltk_data...  
[nltk_data]   Package punkt is already up-to-date!  
[nltk_data] Downloading package punkt_tab to  
[nltk_data]   C:\Users\Mr.Right\AppData\Roaming\nltk_data...  
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
```

Out[7]: True

图 5: NLTK 分词资源检查与 punkt_tab 下载记录

记录显示 punkt 已存在, punkt_tab 被下载并解压, 单元格返回 True。补充资源后, 英文分词程序正常输出结果, 如图3所示。该过程说明, 环境验证应同时检查 Python 依赖库和具体任务所需的数据资源。

7 实验结果分析

通过本次实验可以发现, 中文和英文文本在分词方式上存在明显区别。

英文单词之间通常存在空格, 因此可以较容易地根据空格和标点识别单词边界。例如:

Natural language processing

可以自然划分为:

Natural

language

processing

而中文文本通常连续书写, 词语之间没有天然空格。例如:

自然语言处理是人工智能的重要技术

无法直接根据字符之间的位置判断词语边界, 因此需要借助 jieba 等中文分词工具, 根据词典及概率模型对句子进行合理切分。

此外, 从实验结果中还可以看出, 无论是 jieba 还是 NLTK, 都可能将标点符号作为 token 保留下来。在实际文本分析或机器学习任务中, 通常还需要进行数据清洗, 例如删除标点符号、停用词以及无意义字符。

因此, 自然语言处理中的文本预处理往往不仅包含分词, 还包括文本清洗、标准化以及停用词去除等步骤。

8 思考与分析

8.1 为什么中文不能简单使用空格进行分词？

英文单词之间通常存在天然空格，可以较容易地按照空格进行切分，而中文句子中的词语通常连续书写，不存在明显的词语边界。

例如：

自然语言处理技术

可以理解为：

自然语言 / 处理 / 技术

因此，需要借助中文分词算法，根据词典、统计概率或者语言模型判断词语边界。

8.2 为什么标点符号会出现在分词结果中？

分词程序通常将文本按照字符和词语边界进行切分，并不会自动认定标点符号没有意义，因此中文逗号、句号，以及英文句点等符号都有可能作为独立 token 被保留下来。

是否删除这些标点符号，需要根据具体自然语言处理任务决定。

8.3 如何删除标点符号？

可以在分词完成后对结果进行筛选，例如：

```
1 words = [  
2 word for word in words  
3 if word.strip() and word not in '， 。 ! ? ; : 、 “ ” ‘ ’ ,  
4 ]  
5
```

也可以使用正则表达式实现更加全面的文本清洗。

9 实验总结

通过本次实验，我熟悉了 Anaconda 环境中 Python 依赖库的检查方法和 Jupyter Notebook 的基本操作，并完成了 jieba 中文分词、NLTK 英文分词及 Counter 词频统计。

实验中检查了 jieba 与当前运行环境的对应关系，并补充下载了英文分词所需的资源。环境验证、中文分词和英文分词的截图保存了实际代码及输出，可用于检查依赖与资源是否可用。

从结果中可以看出，中文和英文的词语边界处理方式不同，分词也不会自动完成标点过滤。词频统计反映词元在给定文本中的出现次数，结果的解释需要结合原始文本和是否进行清洗来判断。